المدرسة العليا للتكنولوجيا بسلا
Ecole Supérieure de Technologie de Salé

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Rapport de Projet JEE
Application CRUD de Gestion des Étudiants

Réalisé par : Adil Jeddi, SalahEddine Faraj, Ziad Fettah
Encadrant : Mr.Anisse Khald
Année universitaire : 2025/2026

➢ INTRODUCTION

Ce projet consiste à développer une application web dynamique permettant la gestion des étudiants à travers les opérations CRUD (Create, Read, Update, Delete).

L'application a été développée en utilisant :

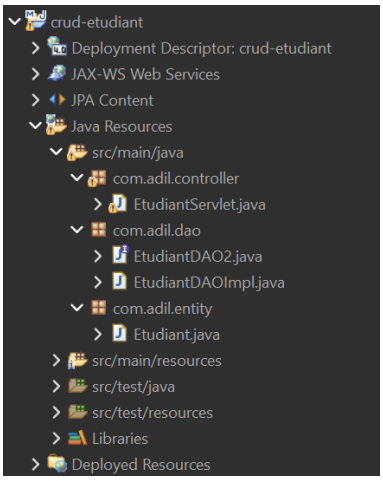
- ✓ Java EE (Servlet & JSP)
- ✓ JPA (Hibernate)
- ✓ Maven
- ✓ MySQL
- ✓ Serveur Tomcat

L'objectif principal est de comprendre l'architecture MVC et le fonctionnement d'une application web basée sur Java EE.

➢ OBJECTIF DU PROJET

- Comprendre l'architecture MVC
- Manipuler les Servlets
- Utiliser JPA pour la gestion de base de données
- Implémenter les opérations CRUD
- Déployer une application web avec Tomcat

➢ ARCHITECTURE DU PROJET



L'application suit le modèle MVC :

1. Model :

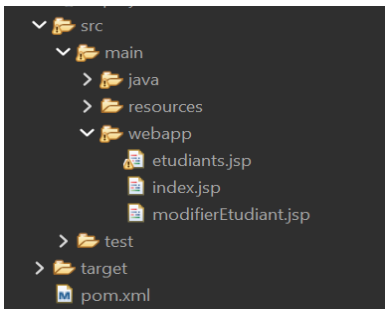
- ✓ Classe `Etudiant`
- ✓ DAO (EtudiantDAO, EtudiantDAOImpl)

2. View :

- ✓ etudiants.jsp
- ✓ Formulaires HTML

3. Controller

- ✓ EtudiantServlet



L'application suit le modèle MVC :

- ✓ `com.adil.entity` → contient les entités JPA
- ✓ `com.adil.dao` → contient les classes d'accès aux données
- ✓ `com.adil.controller` → contient le Servlet
- ✓ `webapp` → contient les fichiers JSP

1) Classe Etudiant.java :

Rôle : Représente la table Etudiant dans la base de données.

```java
package com.adil.entity;

import javax.persistence.*;

@Entity
@Table(name = "etudiant")
public class Etudiant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String nom;
    private String prenom;
    private String email;

    public Etudiant() {}

    public Etudiant(String nom, String prenom, String email) {
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
    }

```

2) Interface EtudiantDAO2

Déclare les méthodes :

- save()
- findAll()
- supprimerEtudiant()

```java
package com.adil.dao;

import com.adil.entity.Etudiant;

public interface EtudiantDAO2 {
    void save(Etudiant e);
    void update(Etudiant e);
    void supprimerEtudiant(int id);
    Etudiant findById(int id);
    List<Etudiant> findAll();
    void delete(Long id);
}

```

3) Classe EtudiantDAOImpl

- **Implémente les méthodes en utilisant JPA.**
- **Faire la communication entre l'application et la base de données.**

```java
31      @Override
32      public List<Etudiant> findAll() {
33          EntityManager em = emf.createEntityManager();
34          try {
35              return em.createQuery("SELECT e FROM Etudiant e", Etudiant.class)
36                      .getResultList();
37          } finally {
38              em.close();
39          }
40      }
41
42      @Override
43      public Etudiant findById(int id) {  // Changé de Long à int pour correspondre à l'interface
44          EntityManager em = emf.createEntityManager();
45          try {
46              return em.find(Etudiant.class, id);
47          } finally {
48              em.close();
49          }
50      }
51
52      @Override
53      public void update(Etudiant e) {
54          EntityManager em = emf.createEntityManager();
55          EntityTransaction tx = em.getTransaction();
56          try {
57              tx.begin();
58              em.merge(e);
59              tx.commit();
60          } catch (Exception ex) {
61              if (tx.isActive()) {
62                  tx.rollback();
```

4) EtudiantServlet :

Contrôler les requêtes HTTP et diriger vers les bonnes pages.

doGet()

- Lister les étudiants
- Supprimer un étudiant
- Afficher le formulaire de modification

doPost()

- Ajouter un étudiant

```java
1   package com.adil.controller;
2
3   import java.io.IOException;
13
14  @WebServlet({"/etudiants", "/supprimer", "/modifier", "/update"})
15  public class EtudiantServlet extends HttpServlet {
16
17      private EtudiantDAO2 dao;
18
19      @Override
20      public void init() {
21          dao = new EtudiantDAOImpl();
22      }
23
24      @Override
25      protected void doGet(HttpServletRequest request, HttpServletResponse response)
26              throws ServletException, IOException {
27
28          String action = request.getServletPath();
29          System.out.println("Action doGet: " + action);
30
31          switch (action) {
32              case "/supprimer":
33                  try {
34                      supprimerEtudiant(request, response);
35                  } catch (Exception e) {
36                      e.printStackTrace();
37                      response.sendRedirect("etudiants?error=suppression");
38                  }
```

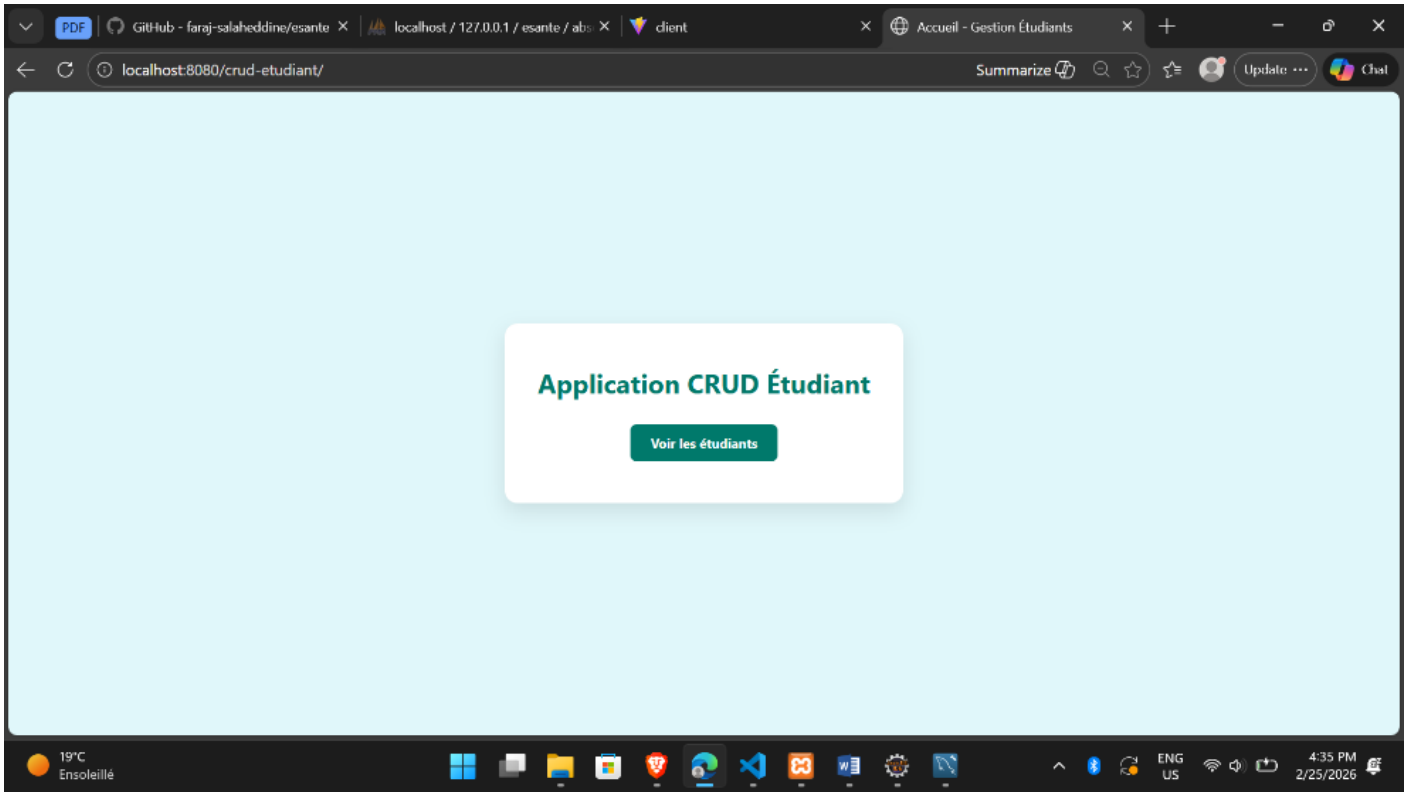
```java
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    String action = request.getServletPath();
    System.out.println("Action doPost: " + action);

    if ("/update".equals(action)) {
        // Mise à jour
        try {
            int id = Integer.parseInt(request.getParameter("id"));
            String nom = request.getParameter("nom");
            String prenom = request.getParameter("prenom");
            String email = request.getParameter("email");

            Etudiant etudiant = new Etudiant(nom, prenom, email);
            etudiant.setId(id);
            dao.update(etudiant);
            response.sendRedirect("etudiants?success=update");
        } catch (Exception e) {
            e.printStackTrace();
            response.sendRedirect("etudiants?error=update");
        }
```

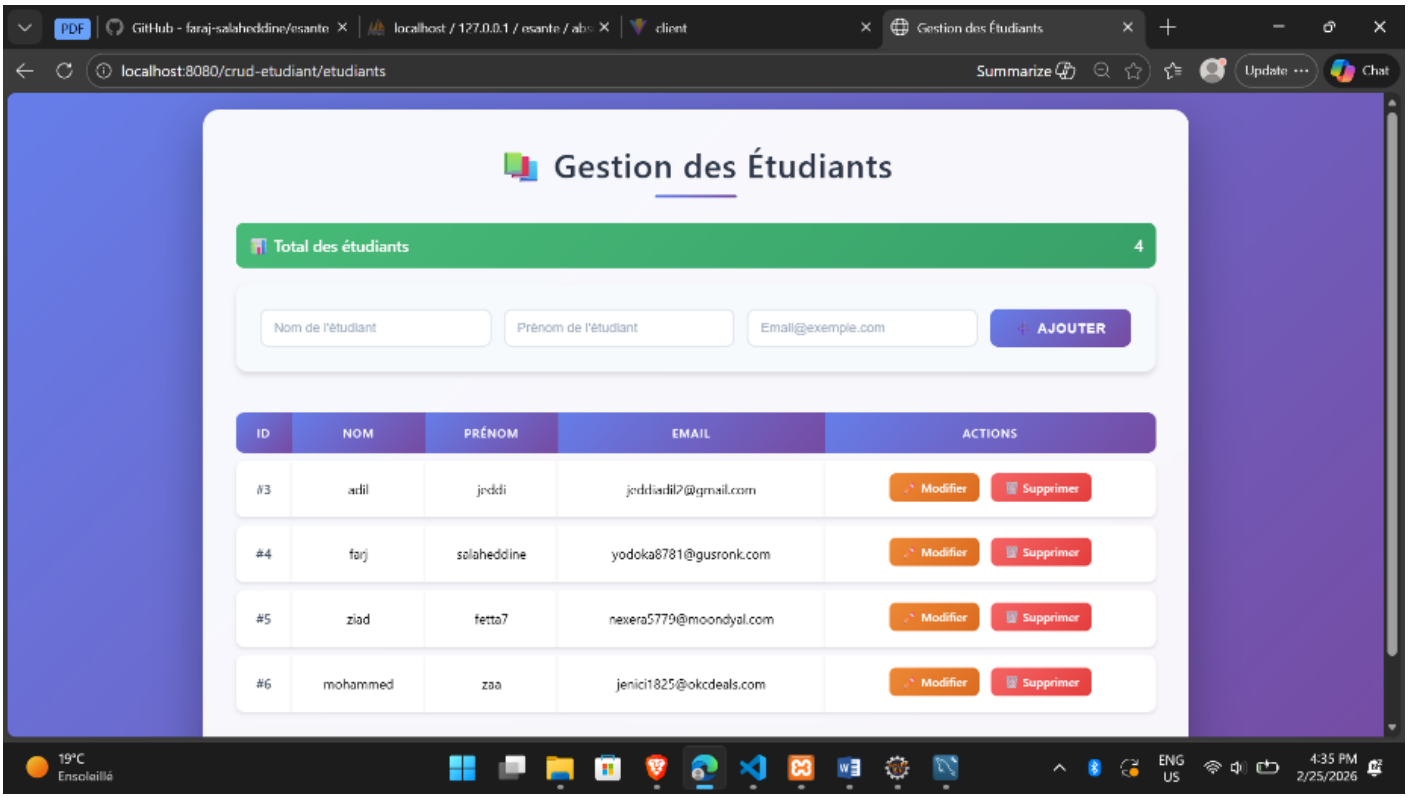
➢ LA VUE (JSP)

- etudiants.jsp

Contient :

- Formulaire d'ajout
- Tableau d'affichage
- Boutons Modifier et Supprimer



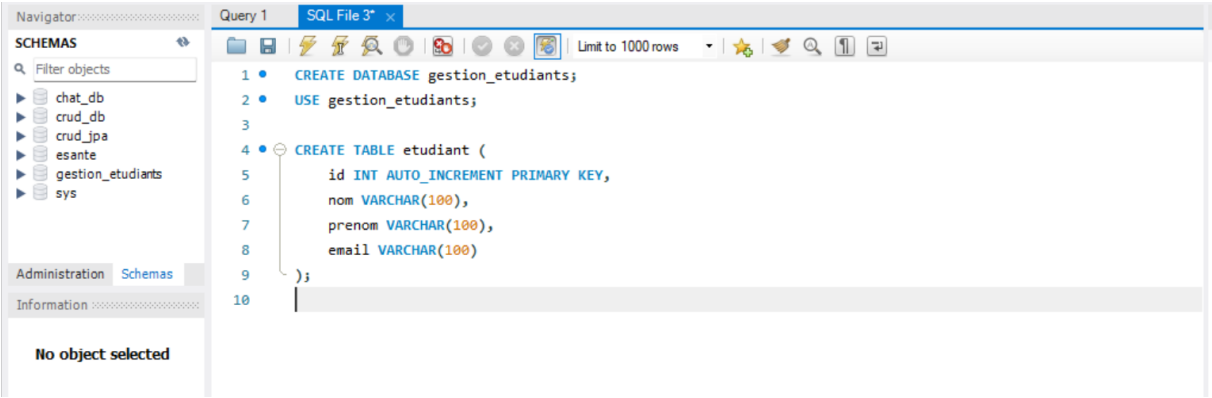


FONCTIONNALITÉS :

✓ Ajouter un étudiant
✓ Afficher la liste
✓ Supprimer un étudiant
✓ Modifier un étudiant

BASE DE DONNÉES :

- La base de données utilisée dans ce projet est MySQL. Elle permet de stocker les informations des étudiants de manière structurée et persistante. La table principale nommée Etudiant contient les champs suivants : *id* (clé primaire auto-incrémentée), *nom*, *prenom* et *email*.



✓ La communication entre l'application Java et la base de données est assurée par JPA (Java Persistence API), qui permet de transformer automatiquement la classe Etudiant en table dans la base de données grâce aux annotations comme @Entity et @Id.

➢ CONCLUSION

Ce projet de gestion des étudiants m'a permis de mettre en pratique les concepts fondamentaux du développement web avec Java EE :

- Le fonctionnement de l'architecture MVC
- La gestion des requêtes HTTP
- L'intégration entre Java et MySQL
- L'utilisation de Maven pour la gestion des dépendances

À travers l'implémentation des opérations CRUD, nous allons compris le fonctionnement de l'architecture MVC, l'utilisation des Servlets et JSP, ainsi que l'intégration de JPA avec une base de données MySQL